Homilivo Chatbot v2: LLM Architecture and Implementation Plan

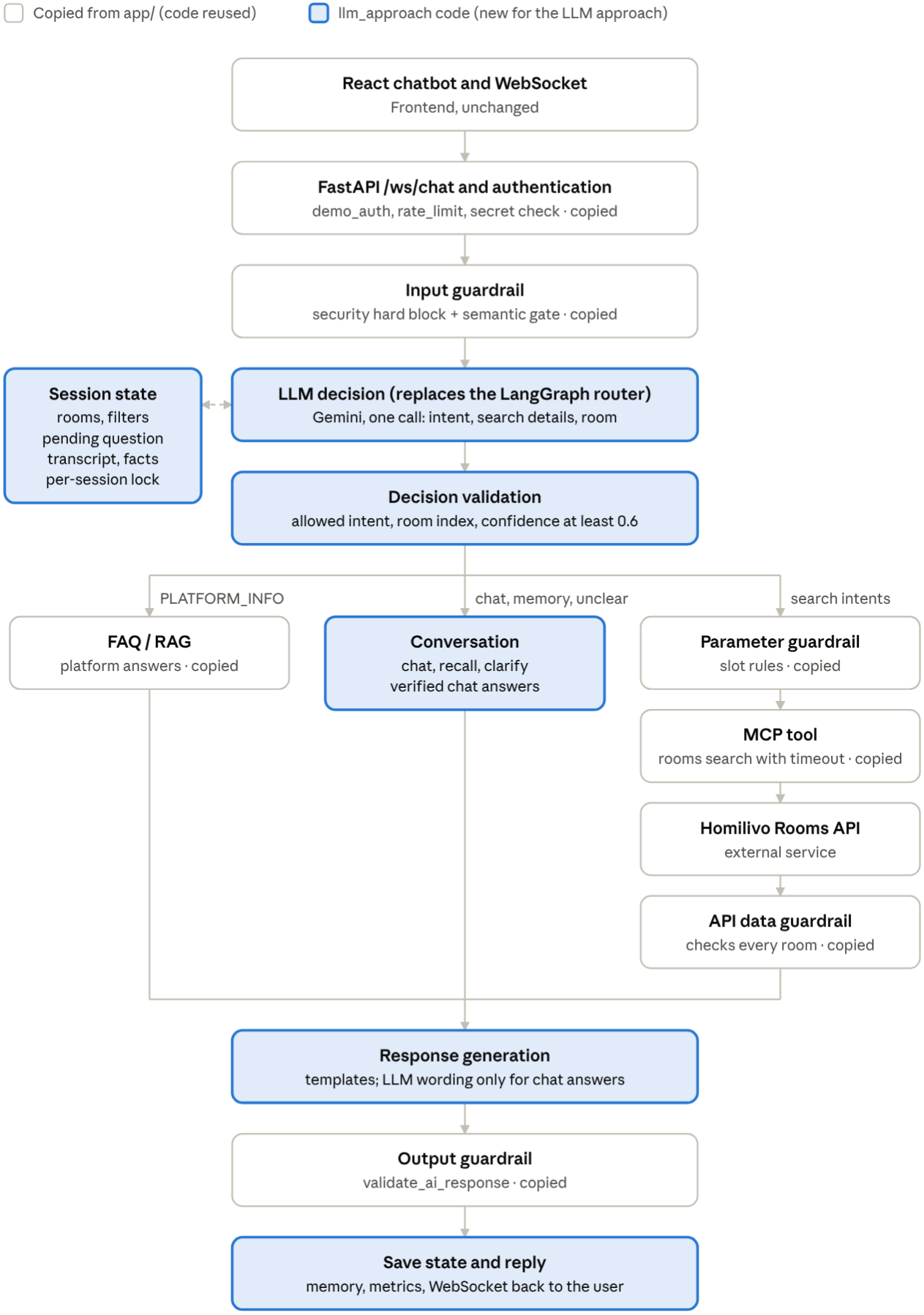
Oct 23, 2026 · @Faizal

v2 keeps the planned architecture stage for stage and replaces only the keyword-based LangGraph router with one validated LLM decision. Every guardrail, the authentication layer, the MCP tool path and the RAG answerer are copied from app/ into llm_approach/ unchanged where possible, on branch experiment/llm-approach_v2.

Target architecture

The shape is the planned architecture: frontend, authentication, input guardrail, a router, three branches, response generation and the output guardrail. Only the router changes: one validated LLM decision picks the branch, and session state feeds it the conversation so far.

v2: one LLM decision replaces the keyword router; guardrails come from app/



v2 pipeline · planned stages, LLM decision in place of the router

A message flows top to bottom. Search intents take the right-hand branch through the parameter guardrail, the MCP tool, the Rooms API and the API data guardrail; platform questions go to FAQ/RAG; chat, memory and unclear messages go to Conversation. Unsafe messages stop at the input guardrail, before any LLM call.

Stage by stage

Every guardrail and security stage of the plan already exists in app/ and is copied in. Only the router is replaced, and the memory and verified-answer parts are new, with no app/ equivalent. Source names come from the security-parity inventory of 21 October; Phase 0 confirms each file and function before anything is copied.

#	Planned stage	llm_approach today	v2 plan	Source in app/
1	React Chatbot, WebSocket	Unchanged frontend, start-frontend-llm.ps1	Unchanged	Frontend repo
2	FastAPI /ws/chat	Own server_app.py	Keep server_app.py; copy the 500-character cap, session id check and token log redaction	api.py: ChatRequest, _run_chat_turn, _normalize_session_id, _RedactTokenFilter
3	Authentication	demo_auth and rate_limit imported through reuse.py	Copy both; add the fail-fast secret check, adapted to require GEMINI_API_KEY and KANGAROO_TOKEN	demo_auth.py, rate_limit.py, api.validate_secrets
4	Input Guardrail	Security hard block imported, plus extra_security.py; semantic gate not used	Copy input_guardrail.py whole; hard block and extra_security before the LLM as today; add the semantic gate with conversation context (decision D2)	input_guardrail.py: is_security_blocked, validate_user_input
5	LangGraph Router	Keyword router replaced by one LLM decision, validators.py and a dispatcher in pipeline.py	**Replaced**: the router node becomes the LLM decision (prompt, schema-checked JSON, confidence threshold), routing by validated intent (decision D1)	Graph shell only; keyword routing not reused
6	FAQ/RAG	actions/platform.py answers PLATFORM_INFO from knowledge-base text	Copy the app/ RAG retriever and answer path; PLATFORM_INFO routes to it	RAG module (name confirmed in Phase 0)
7	Conversation	conversation.py, clarify.py, recall.py, conversation_answerer.py	**Keep** (memory, recall and verified chat answers are new); keep the reused fact-grounding and injection-defence prompt rules	agent_modes: FACT_GROUNDING_RULE, PROMPT_INJECTION_DEFENSE_RULE

#	Planned stage	llm_approach today	v2 plan	Source in app/
8	API Request, Pydantic Extraction	Slots extracted inside the LLM decision (schema.py, Pydantic)	**Keep**: the decision call is the extraction step, so one call, not two	None
9	Parameter Guardrail	Re-implemented as validators.validate_slots	Copy parameter_guardrail.py as the single slot check; validators.py keeps only LLM-specific checks (allowed intent, room index, confidence)	parameter_guardrail.validate_search_arguments
10	MCP Tool	rooms_client.py calls the Rooms API directly	Copy the MCP tool call with its timeout, so search uses the same path as production	agent_modes._call_search_rooms_with_timeout + MCP client
11	Homilivo API	Rooms API GET	Unchanged	None
12	API Guardrail	Not used; only a field allowlist and sanitiser in state.clean_room	Copy api_data_guardrail.py; run validate_room_data on every Rooms API response before rooms reach state or the prompt	api_data_guardrail.validate_room_data, sanitize_room_for_llm_context
13	Response Generation	Templates per action; the LLM only drafts chat-memory answers, verified in code	**Keep**; delimiter neutraliser copied (already wrapped)	agent_modes._neutralize_prompt_delimiters
14	Output Guardrail	validate_ai_response imported through reuse.py	Copy output_guardrail.py unchanged	output_guardrail.validate_ai_response
15	Session state (not drawn)	state.py: rooms, filters, pending question, transcript, facts, queries, per-session lock	**Keep**	None
16	Observability (not drawn)	Own metrics.py and per-turn LLM budget	Keep; align field names with production's per-turn record	observability.record_turn

Reuse strategy

The app/ code is copied, not imported, so v2 runs without importing app/ (the clean start the team lead asked for) while app/ stays untouched. Six rules keep the copies honest:

1. **Copy the whole dependency closure.** A guardrail may import helpers (config, constants, an embedding model for the semantic gate). Phase 0 maps the imports; the copy includes everything the module needs, not just the file.
2. **Change as little as possible.** Only import paths and the lines the new pipeline needs. Every changed line carries a `# v2:` comment.

3. **Provenance header.** Each copied file starts with `Copied from app/<path> at commit <sha>. Changes: none` or the list of changes.
4. **Drift test.** A test compares each unchanged copy with its original (ignoring imports and the header) and fails if either side changes; for a changed copy it checks that the difference matches the recorded change list. A fix made in app/ can then never be silently missing from v2.
5. **Bring the tests along.** The app/ tests for each copied module are copied too and run against the copy.
6. **Retire the shims.** reuse.py is removed once every import points to a local copy.

Target folder layout:

```
llm_approach/
  guardrails/   input_guardrail.py  parameter_guardrail.py
api_data_guardrail.py  output_guardrail.py
  security/     demo_auth.py  rate_limit.py  secrets_check.py
  tools/        mcp_search.py
  rag/          (the app/ FAQ/RAG module and its helpers)
  graph/        router.py   (LangGraph shell, if decision D1 is approved)
  kept as is:   pipeline.py  llm_client.py  providers.py  gemini_limits.py
prompts.py  schema.py
                validators.py  state.py  actions/  conversation_answerer.py
metrics.py  eval/  tests/
```

Design decisions

Six choices shape v2. Each has a recommendation; D1, D2 and D4 need the team lead's confirmation.

#	Decision	Options	Recommendation and why
D1	Orchestration shell	LangGraph StateGraph, or the plain dispatcher in pipeline.py	**LangGraph**, matching the agreed diagram and production. Each node wraps an existing tested pipeline function, so behaviour does not change; the parity test must stay identical. Cost: a refactor of pipeline.py and a LangGraph dependency.

#	Decision	Options	Recommendation and why
D2	Semantic input gate	Block before the LLM, or advise only	**Shadow mode first**: run it before the LLM with the conversation state passed in, log its verdict but do not block, for one full evaluation. Switch to blocking only if it raises 0 false positives on the 144 general messages and the 166 eval items. Reason: a follow-up such as "does it have wifi" looks off-topic without context. The LLM's OUT_OF_SCOPE intent stays as the second line.
D3	Slot checks	Keep validators.validate_slots, or use the app/ parameter guardrail	**One source of truth: the app/ parameter guardrail.** Run both side by side in Phase 3; any rule that exists only in validators.py moves into the copy as a marked v2 change.
D4	Different city with rooms on screen	Carry over old filters, or start fresh	**Start fresh**: a new city clears the old budget, BHK and area; short refinements ("under 15k") keep them. Today handle_refinement merges old filters, so "bhopal me 1bhk" can inherit a Bangalore budget.
D5	LLM calls per message	As today, or more	**Unchanged**: one decision call; a second call only for questions about the chat; the RAG answer path counts inside the same per-turn budget.
D6	Copy or import app/ code	Import through reuse.py, or copy	**Copy**, as the team lead's clean start asks, with the drift test so the copies cannot silently diverge.

Implementation phases

Seven phases, each ending in a check that must pass before the next starts. Durations are estimates in working days and depend on the Gemini quota (about 200 evaluation requests a day) and review time. Each phase is one Claude Code task and one commit.

Phase	Work	Exit criterion	Estimate
0. Branch, baseline, inventory	Commit the counter fix; cut experiment/llm-approach_v2; read-only inventory of every app/ component in the diagram (file, functions, imports, tests, config, external dependencies); record the v1 baseline (prompt 20f328e351bb: pooled 94.0%, ctxY 69.7%)	Inventory reviewed; no code changed	1 day
1. Copy unchanged	Copy the input, output and parameter guardrails, the API data guardrail, demo_auth, rate_limit and the delimiter neutraliser into the new folders with provenance headers; switch imports away from reuse.py; copy their app/ tests; add the drift test. Nothing is wired in yet	All tests pass (1,369 + copied ones); safety 43/43, 0/144, 6/6; parity identical; none of these modules imported from app/	1 day
2. Wire the missing protections	API data guardrail on every Rooms API response; fail-fast secret check at startup; semantic gate in shadow mode	Robustness 7/7 plus new cases; new tests for each guard; shadow report with 0 would-be blocks on the 144 general and 166 eval messages	2 days

Phase	Work	Exit criterion	Estimate
3. Parameter guardrail and MCP tool	Replace validate_slots with the copied parameter guardrail (side-by-side comparison first); route searches through the copied MCP tool call	Invalid-search rate 0; golden_slots at least 96.5% of fields; rooms tests pass	1 day
4. RAG and orchestration shell	Copy the RAG path for PLATFORM_INFO; LangGraph router (if D1); different-city rule (if D4)	Stub scenarios 300/300; parity on unchanged paths; platform answers come from RAG	2 days
5. Live validation	Five evaluation sets on v2; consistency (3 runs); live scenarios; held-out set (if approved)	Every quality gate in the next section	2 days
6. Docs and review	README, pipeline PDF, evaluation report and Overleaf report updated; draft PR from the v2 branch	Team lead and Rudrakash Sir approve	1 day

Quality gates

v2 must hold the v1 experiment's accuracy while adding the protections. The gates below are proposed and need the team lead's confirmation.

Metric	v1 experiment (21 Oct)	v2 gate (proposed)
Intent accuracy, pooled 133	94.0%	at least 92.0% (no drop of more than 2 points)
Intent accuracy, ctxY	69.7%	at least 67.7%
Held-out context set	not built	clearly above the current chatbot (needs approval)
Searches with an invalid filter	0 of 112	0
Unusable decisions	0 of 186	0
Safety: attacks blocked, false alarms, room-text injection	43/43, 0/144, 6/6	unchanged

Metric	v1 experiment (21 Oct)	v2 gate (proposed)
Semantic gate false positives	not used	0 on 144 general and 166 eval messages
Failure handling	7 of 7 clean	7 of 7, plus the new guardrail cases
Consistency across 3 runs	not measured	at least 95% identical decisions
Live scenarios	not run live	every turn passes, or each failure explained
LLM latency p95 per call	3.3 s	3 s target; reported per stage if above
Automated tests	1,369 passing	all passing, including the copied app/ tests

Risks and mitigations

The biggest risk is the semantic gate blocking legitimate follow-ups; shadow mode exists to catch that before it reaches users.

Risk	Effect	Mitigation
Semantic gate misreads follow-ups	"does it have wifi" refused as off-topic	Pass conversation state to the gate; shadow mode before blocking; gate on 0 false positives
Copies drift from app/	A fix lands in one copy only	Provenance header with commit; drift test fails on any unrecorded difference
Dependency closure larger than expected	Bigger install, slower start (for example an embedding model)	Phase 0 maps imports first; load heavy models lazily, once per process
API data guardrail rejects real rooms	Searches come back empty	Run it against a live Rooms API sample in Phase 2 and log each rejection reason

Risk	Effect	Mitigation
LangGraph refactor changes behaviour	Silent regressions	Nodes wrap existing functions; parity and stub-scenario tests must stay identical
Added guards add latency	Slower replies	Measure each stage in metrics.py; keep guards that call no LLM
ctxY gain is partly tuned	Accuracy overstated	Held-out context set written before looking at model answers
Gemini free-tier quota	Runs stop partway	Rolling 24-hour counter, per-day run plan, resume after interruption

Decisions needed from the team lead

- [ ] Branch: name experiment/llm-approach_v2, cut from experiment/llm-approach so the llm_approach/ work carries over (or from DEV)
- [ ] D1: LangGraph shell matching the diagram, or keep the plain dispatcher
- [ ] D2: semantic gate in shadow mode first, blocking only after 0 false positives
- [ ] D4: a different city with rooms on screen starts a fresh search, without the old filters
- [ ] The quality gates above, especially the 2-point tolerance on accuracy
- [ ] Approval to build the held-out context set